

Нейронные сети глубокого обучения

Простые нейросетевые модели

Лекция 4

20 марта 2024 г.

Сукин Иван Андреевич

РХТУ им. Д.И. Менделеева

 sukin.i.a@muctr.ru

Перцептроны

- На основании наблюдений за биологическими нейронами была предложена простейшая модель нейрона — перцептрон:
 - Каждый перцептрон имеет множество входов x_i и один выход y
 - Сигнал на каждом входе либо усиливается, либо ослабляется путём умножения на весовой коэффициент w_i
 - Взвешенные сигналы суммируются
 - Просуммированный результат пропускается через функцию активации f_A и выдаётся на выход
 - Одним из входов может смещение с весом b и $x_b = 1$

Обучение перцептронов

- Перцептрон аппроксимирует функцию $y(x)$ с помощью функции $\bar{y}(x, w, b)$:

$$\bar{y}(x, w, b) = f_A(w^T x + b)$$

- Если известна выборка, состоящая из значений x и соответствующих им y , можно решить задачу аппроксимации, минимизировав различие между y и $\bar{y}$
- Это различие представляется в виде функции потерь $J(y, \bar{y}(x, w, b))$

$$J(w, b) \rightarrow \min_{w, b}$$

Линейная регрессия — пример перцептрона

- Функция активации линейной регрессии:

$$f(z) = z$$

- Функция потерь:

$$J(w, b) = \sum_i L(y_i, x_i, w, b) = \sum_i (y_i - w^T x_i - b)^2$$

- **Сложно!** Эта функция потерь сепарабельна. Что это значит, почему это важно?
- **Сложно!** Вспомните, что эта функция эквивалентна логарифмической функции правдоподобия

Примеры перцептронов Классический перцептрон

Классический перцептрон Розенблатта выполняет бинарную классификацию:

- Функция активации:

$$f_A(z) = \text{sign } z$$

- Функция потерь:

$$J(w, b) = \sum_i L(y_i, x_i, w, b) = \sum_i [y_i - \text{sign}(w^T x_i + b)]^2$$

- Эта функция негладкая

Примеры перцептронов

Гладкий перцептрон

Функцию потерь классического перцептрона можно сгладить:

- Функция активации:

$$f_A(z) = z$$

- Функция потерь:

$$J(w, b) = \sum_i L(y_i, x_i, w, b) = \sum_i \max(0, -y_i(w^T x_i + b))$$

- **Сложно!** Докажите, что эта функция гладкая по w

Примеры перцептронов Логистическая регрессия

Логистическая регрессия выполняет более мягкую двоичную классификацию:

- Функция активации:

$$f_A(z) = \frac{1}{1 + e^{-z}}$$

- Функция потерь — максимальное логарифмическое правдоподобие:

$$\begin{aligned} J(w, b) &= \sum_i L(y_i, x_i, w, b) = \\ &= - \sum_i \left[y_i \ln \frac{1}{1 + e^{-w^T x + b}} + (1 - y_i) \ln \left(1 - \frac{1}{1 + e^{-w^T x + b}} \right) \right] \end{aligned}$$

Примеры перцептронов Метод опорных векторов

В методе опорных векторов строится опорная гиперплоскость, разбивающая исходные данные на два класса:

- Функция активации:

$$f_A(z) = z$$

- Функция потерь кусочно-линейная:

$$J(w, b) = \sum_i L(y_i, x_i, w, b) = \sum_i \max(0, -y_i(w^T x_i) + 1)$$

Недостатки перцептронов

- Перцептрон — ограниченный метод аппроксимации
- Перцептрон не может распознать важные геометрические свойства фигур, например, связность
- Перцептрон не будет работать, если между классами данных нельзя построить гиперплоскость
 - Простейший пример — задача XOR

Обучение XOR

- Обучающая выборка:

X	y
$[-1, -1]$	-1
$[-1, 1]$	1
$[1, -1]$	1
$[1, 1]$	-1

- Возьмём классический перцептрон:

$$\bar{y} = \text{sign}(w^T x + b)$$

Обучение XOR

Функция потерь:

$$J(w, b) = \sum_i [y_i - \text{sign}(w^T x_i - b)]^2 =$$

$$= [-1 - \text{sign}(-w_1 - w_2 + b)]^2 + [1 - \text{sign}(-w_1 + w_2 + b)]^2 + \\ + [1 - \text{sign}(w_1 - w_2 + b)]^2 + [-1 - \text{sign}(w_1 + w_2 + b)]^2$$

- Минимум этой функции достигается при $w_1 = w_2 = b = 0$
- Перцептрон просто выдаёт тождественный ноль

Нелинейность

Что делать с тем, что перцептрон не может аппроксимировать даже такую простую функцию как XOR?

- Первый шаг — использование принципиально нелинейной функции активации
- Раньше использовались сигмoиды и гиперболические тангенсы
- Сейчас чаще всего применяется функция «немного отличающаяся от линейной» — ReLU:

$$\text{relu}(z) = \max(0, z)$$

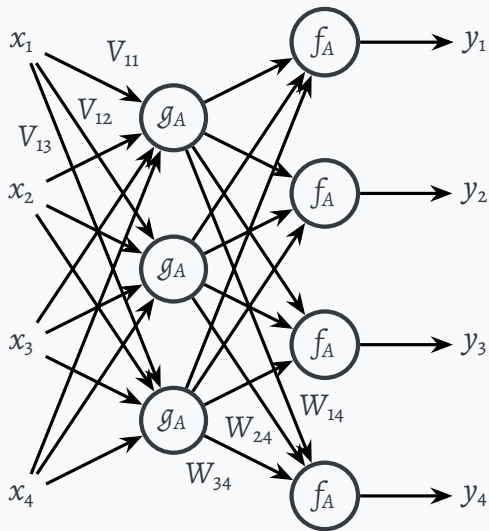
Скрытые слои

Достаточно ли одной нелинейной функции активации?

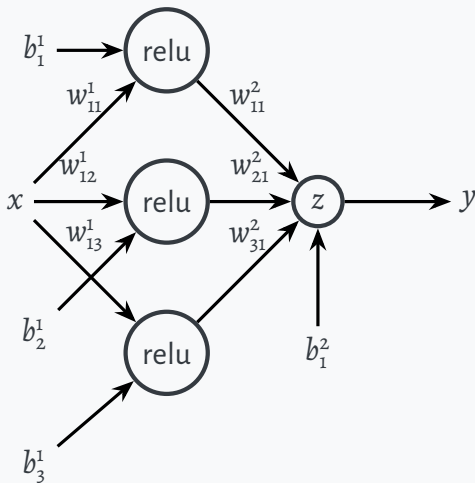
- Часто нужно преобразовать исходное пространство
 - Взгляните на пространство X для XOR
- Можно взять ещё M перцептронов, пропустить x через каждый из них, а результат послать в качестве входа на исходный перцептрон
- Получим «скрытый слой» перцептронов и дополнительные $N \times (M + 1)$ параметров (V, c)
- В математическом выражении:

$$\bar{y} = f_A \left(w^T g_A (Vx + c) + b \right)$$

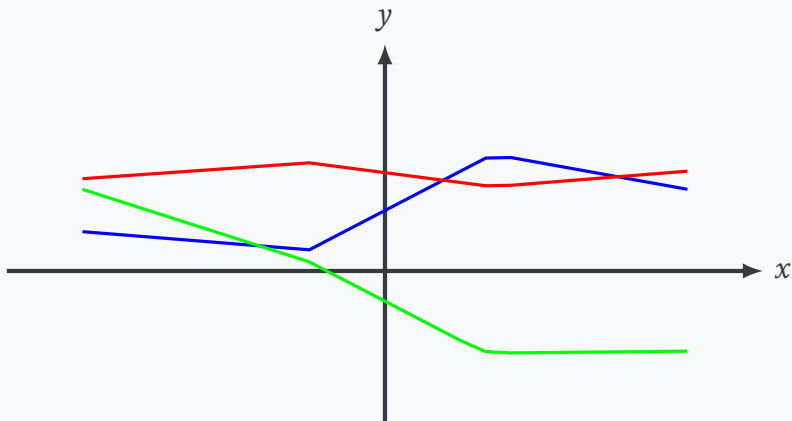
Двухслойный перцептрон



Простой двухслойный перцептрон

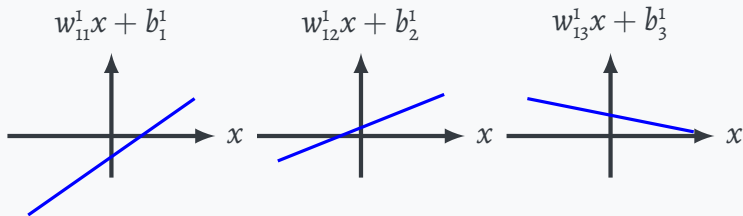


Аппроксимируемые функции

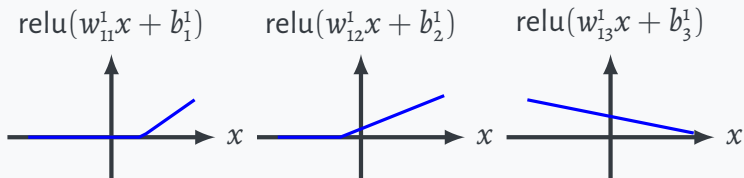


Сколько здесь линейных участков?

Входы перцептронов первого слоя

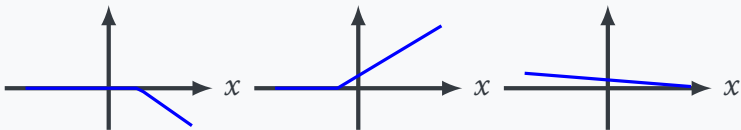


Выходы перцептронов первого слоя



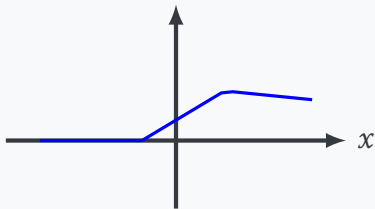
Входы перцептрона второго слоя

$$w_{11}^2 \operatorname{relu}(w_{11}^1 x + b_1^1) \quad w_{21}^2 \operatorname{relu}(w_{12}^1 x + b_2^1) \quad w_{31}^2 \operatorname{relu}(w_{13}^1 x + b_3^1)$$

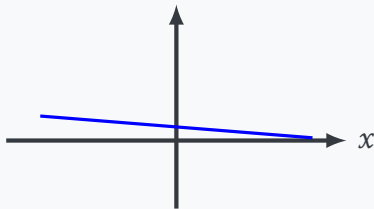


Входы перцептрона второго слоя

$$w_{11}^2 \operatorname{relu}(w_{11}^1 x + b_1^1) + w_{21}^2 \operatorname{relu}(w_{12}^1 x + b_2^1)$$

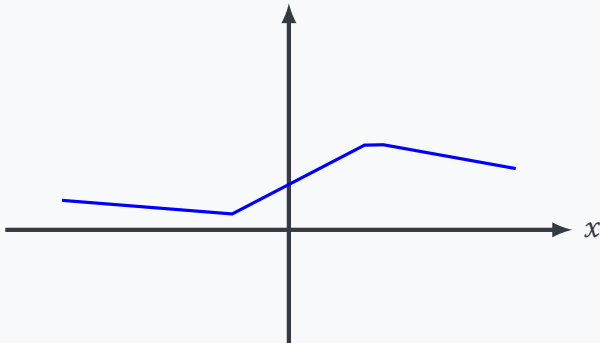


$$w_{31}^2 \operatorname{relu}(w_{13}^1 x + b_3^1)$$

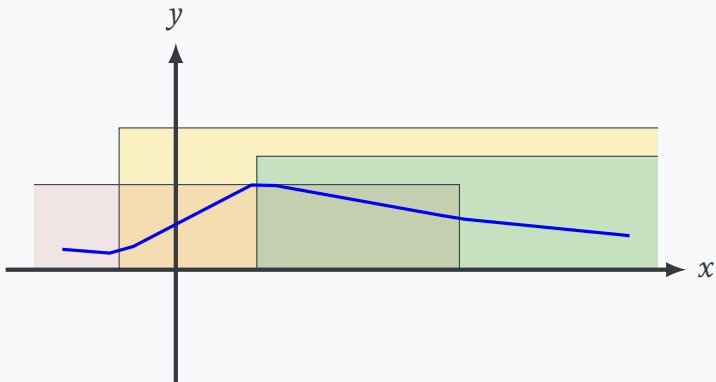


Выход перцептрона второго слоя

$$b_1^2 + w_{11}^2 \operatorname{relu}(w_{11}^1 x + b_1^1) + w_{21}^2 \operatorname{relu}(w_{12}^1 x + b_2^1) + w_{31}^2 \operatorname{relu}(w_{13}^1 x + b_3^1)$$



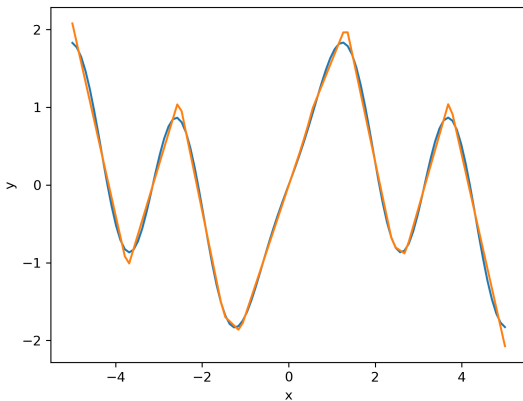
Области активации перцептронов первого слоя



Теорема об универсальной аппроксимации

- Простой двухслойный перцептрон с достаточным количеством элементарных перцептронов на первом слое может с любой заранее заданной точностью аппроксимировать любую непрерывную функцию на компактном множестве
 - Функции активации перцептронов первого слоя — ReLU
 - Функция активации перцептрона второго слоя — тождественная

Теорема об универсальной аппроксимации



Обучение простого перцептрона

- Рассмотрим сепарабельную функцию потерь:
 $J(W, b) = \sum_i L(y_i, x_i, W, b)$
- Для простого двухслойного перцептрона она имеет вид:

$$J(w^2, w^1, b^2, b^1) = \sum_i \left(y_i - (w^2)^T \underbrace{\text{relu}(w^1 x_i + b^1)}_{\text{вектор}} - b_1^2 \right)^2$$

- Найдём градиент этой функции (производные по каждой переменной)

Обучение простого перцептрона

- Производная по b_1^2 :

$$D_{b^2}J = -2 \sum_i \underbrace{(y_i - (w^2)^T \text{relu}(w^1 x_i + b^1) - b_1^2)}_{\text{скаляр}}$$

- Градиент по w^2 :

$$D_{w^2}J = -2 \sum_i \underbrace{\text{relu}(w^1 x_i + b^1)}_{\text{вектор}} \underbrace{(y_i - (w^2)^T \text{relu}(w^1 x_i + b^1) - b_1^2)}_{\text{скаляр}}$$

- Обозначим:

$$L_i = y_i - (w^2)^T \text{relu}(w^1 x_i + b^1) - b^2$$

Обучение простого перцептрона

- Производная ReLU — функция Хевисайда $H(z)$:

$$H(z) = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0 \end{cases}$$

- Градиент J по b^1 :

$$D_{b^1}J = -2 \sum_i \underbrace{w^2 \circ H(w^1 x_i + b^1)}_{\text{поэлементно}} L_i$$

- Градиент J по w^1 :

$$D_{w^1}J = -2 \sum_i \underbrace{w^2 \circ H(w^1 x_i + b^1)}_{\text{поэлементно}} \otimes x_i L_i$$

Градиентный спуск для простого перцептрона

1. Зададим параметры:
 - $\varepsilon > 0$ — скорость обучения
 - $\delta > 0$ — точность
2. Зафиксируем некие начальные значения $w^2[0]$, $w^1[0]$, $b^2[0]$, $b^1[0]$, $j = 0$
3. Цикл
 - 3.1. Вычислим для всех i :

$$L_i[j] = y_i - (w^2[j])^T \text{relu}(w^1[j]x_i + b^1[j]) - b^2[j]$$

$$V_i[j] = w^2[j] \circ H(w^1[j]x_i + b^1[j])$$

Градиентный спуск для простого перцептрона

3. Цикл (продолжение):

3.2. Вычислим производные (константа -2 не важна):

$$D_{b^2}J[j] = \sum_i L_i[j]$$

$$D_{w^2}J[j] = \sum_i \text{relu}(w^1[j]x_i + b^1[j])L_i[j]$$

$$D_{b^1}J[j] = \sum_i V_i[j]L_i[j]$$

$$D_{w^1}J[j] = \sum_i V_i[j]L_i[j] \otimes x_i$$

3.3. Градиент равен:

$$DJ[j] = [D_{b^2}J[j], D_{w^2}J[j], D_{b^1}J[j], D_{w^1}J[j]]$$

Градиентный спуск для простого перцептрона

3. Цикл (продолжение)

3.4. Если некоторая норма $|DJ[j]|$ меньше δ , выходим из цикла, результатом обучения является вектор $[b^2[j], w^2[j], b^1[j], w^1[j]]$

3.5. В противном случае вычисляем:

$$[b^2[j+1], w^2[j+1], b^1[j+1], w^1[j+1]] = [b^2[j], w^2[j], b^1[j], w^1[j]] + \varepsilon DJ[j]$$

и переходим к следующей итерации с $j = j + 1$

Проблемы градиентного спуска

- Обучающая выборка слишком большая
 - На каждом этапе можно выбирать небольшой «мини-пакет»
- Шаг в сторону, противоположную градиенту, получается слишком большим
 - Можно на каждом шаге уменьшать скорость обучения ϵ
- Функция ReLU нечувствительна к отрицательным значениям
 - Можно использовать Leaky ReLU
- Неудобно каждый раз выводить формулы для градиента при разных структурах сети
 - Можно использовать алгоритм обратного распространения

О лекциях

- Материал лицензирован под [CC BY SA 4.0](#)
- Правки и предложения приветствуются в виде пулл-реквестов по адресу <https://codeberg.org/fmjr/muctr>